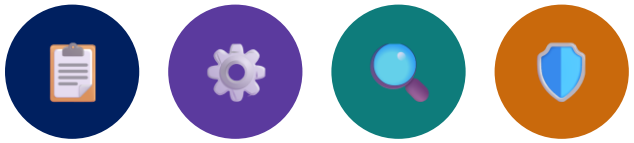
MODULE 17

JUnit Testing Fundamentals

Learn the core concepts and best practices of unit testing with JUnit.



TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 17 OVERVIEW

Learn the core concepts and best practices of unit testing with JUnit.

- Unit tests catch defects early, enable safe refactoring, and act as living documentation. This module covers JUnit 5 architecture, lifecycle, assertions, and test coverage.

DURATION & LAB

1 Hour Theory

JUnit architecture, lifecycle, assertions, coverage.

2 Hours Lab

JUnit Testing with AI Assistance.

Scenario

Test the business logic of a CustomerService.

WRITE TESTS
JUnit 5 annotations, assertions, lifecycle

AUTOMATE & ANALYZE
Parameterized tests, coverage measurement, and test refinement

THE PAYOFF
Reliable, maintainable, well-tested code

KEY TAKEAWAY Well-tested code is easier to refactor, catches defects early, and builds confidence in every release.

WHY UNIT TESTING MATTERS

Unit testing is the foundation of reliable, maintainable, and high-quality software.

Catch Bugs Early

- Find defects at the source before they impact users.

Enable Safe Changes

- Refactor and add new features with confidence.

Improve Code Quality

- Write cleaner, modular code that's easier to test.

Save Time and Cost

- Reduce debugging time and prevent costly defects.

Support Teamwork

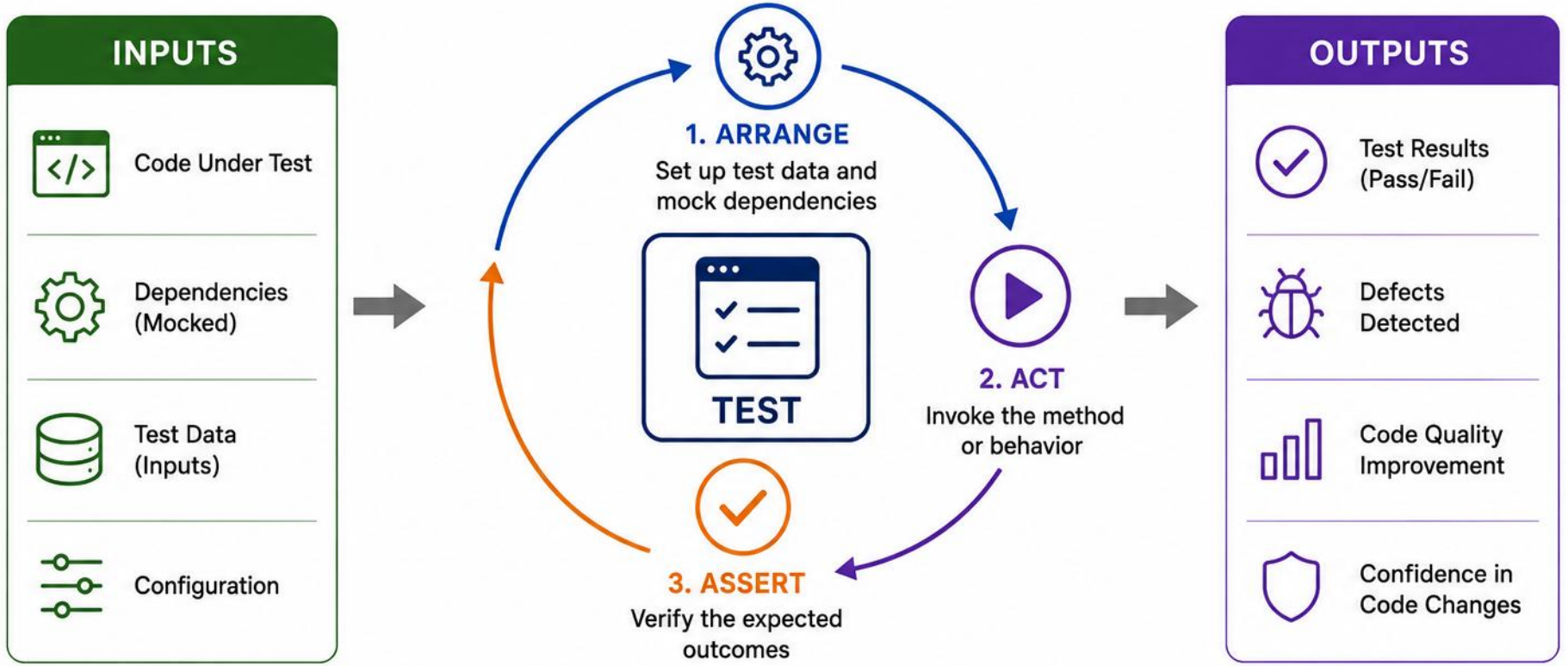
- Tests serve as documentation and support collaboration.

Build Confidence

- Ensures releases behave as expected.

KEY TAKEAWAY Unit testing is the foundation of reliable, maintainable, and high-quality software.

UNIT TESTING



KEY BENEFITS

- Early Bug Detection
- Improved Code Quality
- Faster Development
- Safe Refactoring
- Maintainable Code

ARRANGE-ACT-ASSERT IN CODE

Structure each test into three clear phases so intent is obvious at a glance.

PHASE	WHAT HAPPENS	IN THIS TEST
Arrange	Set up objects, inputs, and dependencies	new Order(150.00), new DiscountPolicy(...)
Act	Call exactly one method or behavior	policy.applyTo(order)
Assert	Verify the outcome matches expectations	assertEquals(135.00, total, 0.001)
Comments	Mark each phase so intent reads at a glance	// Arrange, // Act, // Assert
One focus	Keep each test verifying one behavior	split multi-behavior checks into separate tests

COMPLETE EXAMPLE (AAA-STRUCTURED)

```
@Test
void shouldApplyDiscount_whenOrderExceeds() {
    // Arrange
    Order order = new Order(150.00);
    DiscountPolicy policy =
        new DiscountPolicy(0.10, 100.00);

    // Act
    double total = policy.applyTo(order);

    // Assert
    assertEquals(135.00, total, 0.001);
}
```

WHY AAA HELPS

- Comments or blank lines make the three phases visually obvious, even without extra tooling.
- Keeps setup, the action under test, and verification cleanly separated — easier to read and debug.
- Works for state-based assertions and exception-based tests alike; only the Act/Assert lines change.

KEY TAKEAWAY Arrange-Act-Assert turns a wall of code into a test anyone can read in seconds.

TRY IT YOURSELF — EXERCISE 5: AAA SERVICE TESTS PLAN

Reinforces: Arrange–Act–Assert outlines for service tests without deep Mockito yet.

STEP	WHAT TO DO
Goal	Outline AAA steps for two CustomerService tests
File	aaa-service-plan.md (in examples/module-17-exercises/notes/)
Tests	create valid customer; activate prospect Ravi (CUS-1002)
Boundary	Plan collaborators; leave deep mocking for Lab 18
Deliverable	Two AAA outlines with expected asserts named
Reference	exercises/exercise-05-aaa-service-tests-plan.md

```
$ cat aaa-service-plan.md
Arrange: repo + validator fixtures
Act: service.create(...) / activate("CUS-1002")
Assert: status ACTIVE + interaction notes
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-aaa-service-tests-plan.md.

TEST PYRAMID OVERVIEW

A common guideline for a balanced test strategy: keep most feedback fast and close to the code, while using enough integration and end-to-end tests to verify important boundaries and user journeys.

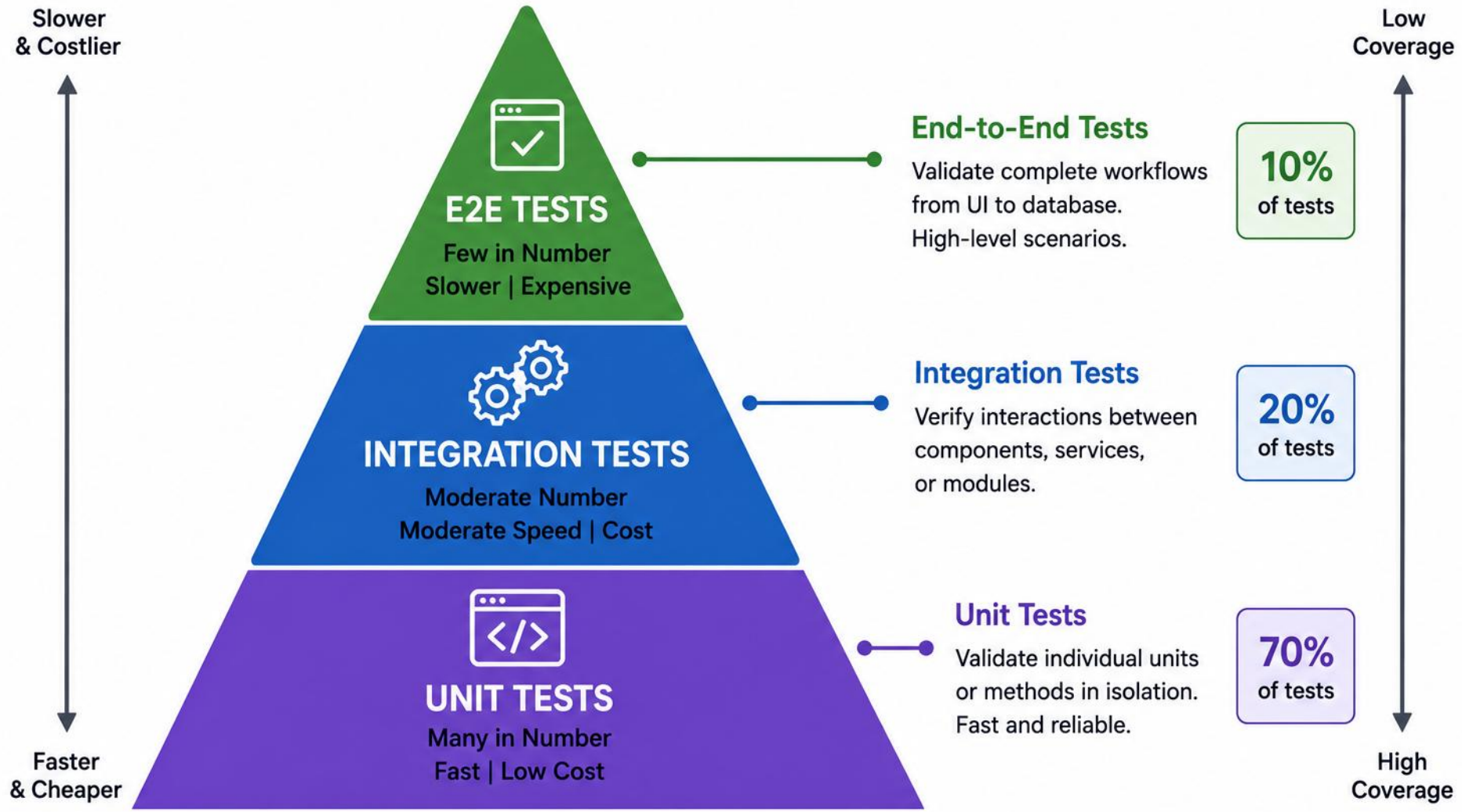
LAYER	WHAT IT VERIFIES	SPEED / COST
`Unit Tests (Many)`	Unit tests verify a small unit of observable behavior with controlled dependencies	Fastest / Low Cost
`Integration Tests (Some)`	Interaction between multiple components or modules	Slower / Medium Cost
`End-to-End Tests (Few)`	The complete system from end to end	Slow / High Cost

GUIDELINE, NOT A RIGID FORMULA

- The test pyramid is a common guideline: keep most feedback fast and close to the code, while using enough integration and end-to-end tests to verify important boundaries and user journeys.
- Not every unit needs a test, and integration-heavy systems may need more component tests; UI-heavy systems may use a testing trophy or other balance.
- Test distribution should reflect risk — use the right type of test at the right level, not a fixed ratio.

KEY TAKEAWAY A balanced test pyramid catches issues early, gives faster feedback, and keeps maintenance cost low.

TEST PYRAMID



JUNIT ARCHITECTURE

JUnit provides a lightweight, extensible architecture built around a few core components.

Test Class

Contains test methods + annotations.

JUnit Platform

Discovery and execution foundation; used by IDEs and build tools via the Launcher.

JUnit Jupiter

Programming and extension model for JUnit 5 tests.

Test Engine

Executes a test model; the Jupiter Engine runs JUnit 5 tests.

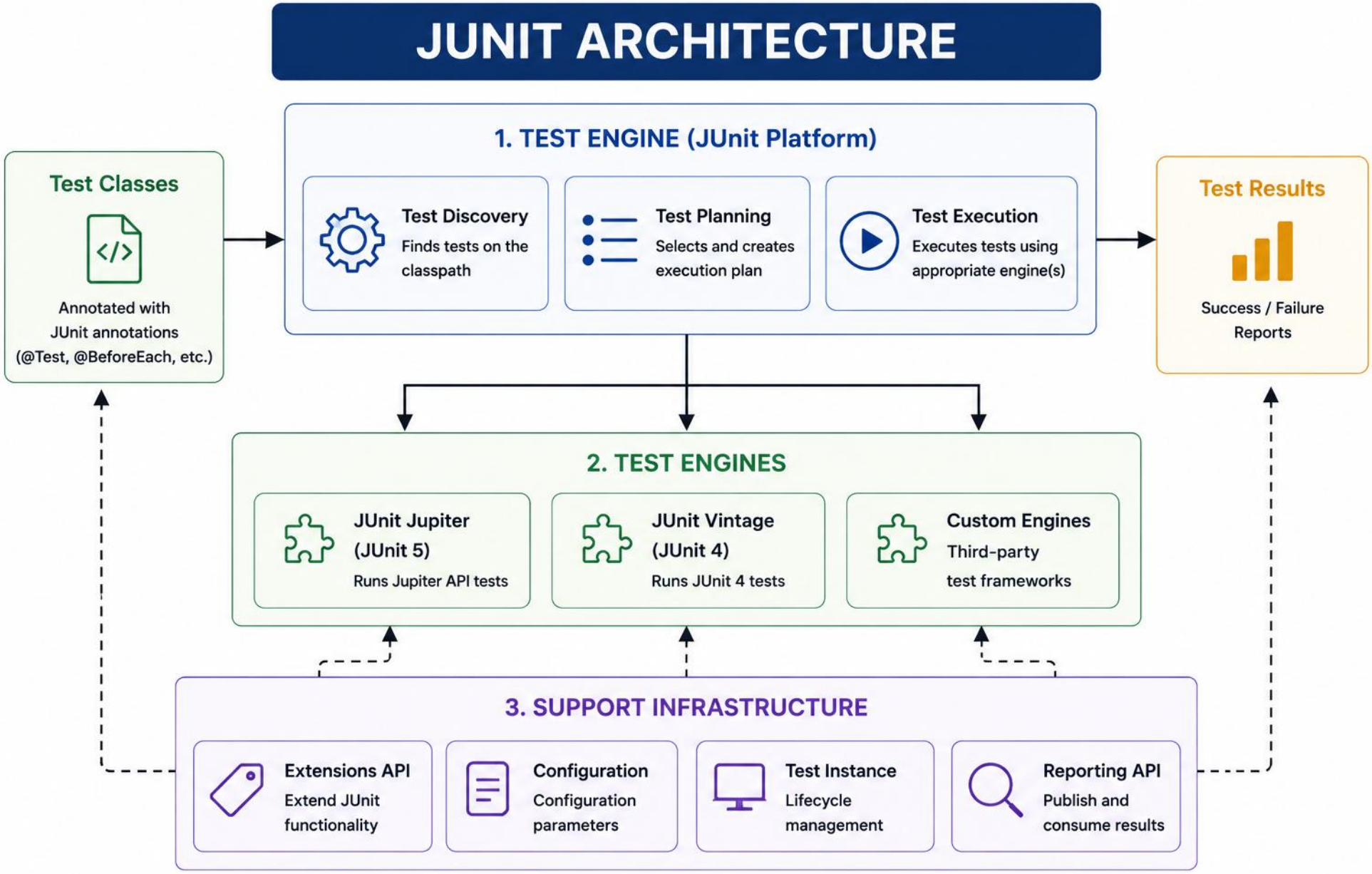
Test Reports

Collects and presents results; listeners consume execution events.

HOW IT FITS TOGETHER

- Flow: IDE / Maven / Gradle → JUnit Platform Launcher → Jupiter Engine → Test Results.
- JUnit 5 is a family of three sub-projects: JUnit Platform (foundation), JUnit Jupiter (JUnit 5 programming/extension model), and JUnit Vintage (legacy support for running older JUnit 3/4 tests).

KEY TAKEAWAY JUnit's layered architecture separates test discovery, execution, and reporting for a clean, extensible test framework.



JUNIT LIFECYCLE AND CORE ANNOTATIONS

JUnit follows a well-defined lifecycle, driven by a small set of core annotations, to run tests consistently.

ANNOTATION	TIMING	USE CASE
`@BeforeAll`	Runs once before all tests in the class	Expensive immutable fixture (e.g., shared parser config)
`@BeforeEach`	Runs before each test method	Set up test environment
`@Test`	The test method itself	Contains assertions to verify behavior
`@AfterEach`	Runs after each test method	Cleanup after the test
`@AfterAll`	Runs once after all tests in the class	Global cleanup

LIFECYCLE ORDER + STATIC RULE (FIX E)

- For each test method: @BeforeEach → @Test → @AfterEach; @BeforeAll/@AfterAll run once each and are normally static unless per-class lifecycle is enabled: @TestInstance(Lifecycle.PER_CLASS).

@BEFOREALL SETUP (FIX F)

- Use @BeforeAll only for expensive, immutable setup (fixture, parser config, one-time data) — never a live DB connection. Prefer lightweight @BeforeEach setup; label any DB/network resource as an integration test.

MORE ANNOTATIONS + ADVANCED (OPTIONAL): @RepeatedTest

- @DisplayName, @Disabled, @Tag, @Timeout add readability/control. @RepeatedTest may expose flakiness — investigate timing, shared state, randomness, concurrency, and external dependencies.

KEY TAKEAWAY A consistent lifecycle ensures clean setup and teardown, keeping tests isolated and reliable.

JUNIT LIFECYCLE IN ACTION

One class, two tests — see the exact call order @BeforeAll through @AfterAll produce.

CALL ORDER	METHOD	WHEN IT RUNS
1	initAll()	Once, before any test in the class (must be static)
2 and 5	init()	Before every @Test method
3 and 6	a @Test method	The test itself
4 and 7	tearDown()	After every @Test method
8	tearDownAll()	Once, after all tests (must be static)

COMPLETE EXAMPLE (EXECUTION ORDER)

```
class OrderServiceTest {

  @BeforeAll static void initAll() { }
  @BeforeEach void init() { }

  @Test void createsOrder() { }
  @Test void cancelsOrder() { }

  @AfterEach void tearDown() { }
  @AfterAll static void tearDownAll() { }
}
```

READING THE ORDER

- @BeforeAll and @AfterAll run once per class and must be static, unless @TestInstance(Lifecycle.PER_CLASS) is used.
- @BeforeEach and @AfterEach wrap every single @Test method, keeping each test's fixture fresh.
- For 2 tests: initAll → init → test1 → tearDown → init → test2 → tearDown → tearDownAll.

KEY TAKEAWAY A predictable lifecycle order is what makes @BeforeEach/@AfterEach safe for isolating every test.

ASSERTIONS IN JUNIT

Assertions verify that the actual results of your code match the expected results.

ASSERTION	DESCRIPTION	EXAMPLE
`assertEquals(expected, actual)`	Verifies logical equality using equals() — see assertSame note below	assertEquals(5, calc.add(2,3))
`assertTrue(condition)`	Verifies that a condition is true	assertTrue(user.isActive())
`assertNull / assertNotNull(obj)`	Verifies an object is/isn't null	assertNotNull(user.getName())
`assertArrayEquals(expected, actual)`	Verifies that two arrays are equal	assertArrayEquals(new int[]{1,2}, r)
`assertThrows(Exception.class, ...)`	Verifies the expected exception is thrown	assertThrows(IllegalArgumentException.class, ...)

COMPLETE ASSERTTHROWS EXAMPLE

```
IllegalArgumentException exception =
    assertThrows(
        IllegalArgumentException.class,
        () -> service.activateCustomer("missing")
    );
assertEquals(
    "Customer not found",
    exception.getMessage()
);
```

ASSERTION PRECISION, MESSAGES & MORE

- assertEquals compares logical equality using equals(). assertSame checks whether two references point to the same object.
- Add a message when the expected behavior is not already obvious from the test name and values:

```
assertEquals(expected, actual,
    () -> "Unexpected result for customer " + customerId);
```

- Also available: assertAll, assertIterableEquals, assertDoesNotThrow. Use assertTimeout carefully — timing-based tests can be unstable.

KEY TAKEAWAY When an assertion fails, the test fails — clear, specific assertions make failures fast to diagnose.

TEST ANNOTATIONS

ANNOTATION	PURPOSE	SCOPE	EXECUTION TIME	EXAMPLE

@BeforeAll	Run once before all test methods	Per class	Once before any test runs	`@BeforeAll`
				`static void setupAll() {`
				`  // initialization`
				`}`

@BeforeEach	Run before each test method	Per test	Before each @Test method	`@BeforeEach`
				`void setUp() {`
				`  // set up`
				`}`

@Test	Marks a method as a test	Per test	When test is executed	`@Test`
				`void testMethod() {`
				`  // test logic`
				`}`

@AfterEach	Run after each test method	Per test	After each @Test method	`@AfterEach`
				`void tearDown() {`
				`  // clean up`
				`}`

@AfterAll	Run once after all test methods	Per class	Once after all tests complete	`@AfterAll`
				`static void cleanupAll() {`
				`  // cleanup`
				`}`

TRY IT YOURSELF — EXERCISE 3: MEANINGFUL ASSERTS

Reinforces: assertEquals / assertTrue with intent — not assertNotNull-only tests.

STEP	WHAT TO DO
Goal	Rewrite a weak assertNotNull-only test into meaningful asserts
File	meaningful-asserts.md (in examples/module-17-exercises/notes/)
Weak	assertNotNull(customer) — passes even when status/email are wrong
Strong	Assert id, status, and email (or business rule) explicitly
Deliverable	Before/after snippet + why the weak test can hide defects
Reference	exercises/exercise-03-meaningful-asserts.md

```
$ cat meaningful-asserts.md
// weak: assertNotNull(c);
assertEquals("CUS-1001", c.getId());
assertEquals(Status.ACTIVE, c.getStatus());
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-meaningful-asserts.md.

PARAMETERIZED TESTS

Run the same test logic multiple times with different input values, reducing duplication.

ANNOTATION	DESCRIPTION	EXAMPLE
@ValueSource	Provides a list of simple values	@ValueSource(ints = {1, 2, 3})
@CsvSource	Provides comma-separated values	@CsvSource({"1,2","3,4"})
@MethodSource	Uses a method that returns a stream/list	@MethodSource("dataProvider")
@EnumSource	Provides values from an Enum	@EnumSource(Day.class)
@CsvFileSource	Reads data from a CSV file	@CsvFileSource(resources="/data.csv")

COMPLETE EXAMPLE (CSV-DRIVEN, CRM DOMAIN)

```
@ParameterizedTest
@CsvSource({
    "PROSPECT, ACTIVE, true",
    "ACTIVE, PROSPECT, false"
})
void shouldValidateStatusTransition(
        CustomerStatus current,
        CustomerStatus requested,
        boolean expected) {
    assertEquals(
        expected,
        validator.isAllowed(current, requested)
    );
}
```

BENEFITS, WHEN TO USE, AND CSV NOTES

- Improve test coverage with less code; each value is executed separately with clear pass/fail results.
- Use parameterized tests when the behavior is the same and only data varies. Use separate named tests when scenarios represent materially different business behavior or failure reasons.
- Lab note: CSV data may need quoting for commas, empty strings, nulls, and spaces in @CsvSource values.

KEY TAKEAWAY Use parameterized tests for business rules, calculations, and validations that depend on multiple input values.

PARAMETERIZED TESTS

1. Test Method

Single test method runs multiple times with different inputs.

2. Data Source

- @ValueSource
- @CsvSource
- @MethodSource
- @EnumSource
- @ArgumentsSource

3. Parameter Injection

JUnit provides input parameters to the test method for each run.

4. Test Execution

Test runs once for each set of parameters.

5. Results

Results are reported for each execution separately.

EXAMPLE

```java
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
   @ParameterizedTest
   @ValueSource(ints = { 1, 2, 3, 4, 5 })
   void testSquare(int input) {
      Calculator calc = new Calculator();
      assertEquals(input * input, calc.square(input));
   }
}
```

HOW IT WORKS

Iteration	Input (input)	Expected (input * input)	Actual (calc.square(input))	Result
1	1	1	1	✓
2	2	4	4	✓
3	3	9	9	✓
4	4	16	16	✓
5	5	25	25	✓

COMMON SOURCES

@ValueSource
Use for primitive values and strings.

@CsvSource
Use for multiple values in CSV format.

@MethodSource
Use a method that returns a Stream, Collection, or array.

@EnumSource
Use all constants from an enum.

@ArgumentsSource
Use custom ArgumentsProvider.

TRY IT YOURSELF — EXERCISE 2: CSVSOURCE TABLE DESIGN

Reinforces: parameterized tests — one assertion path, many CRM status inputs.

STEP	WHAT TO DO
Goal	Design a @CsvSource table for status / expected outcome rows
File	csvsource-table.md (in examples/module-17-exercises/notes/)
Columns	inputStatus, expectedActive, notes
Cases	Include ACTIVE, PROSPECT, and at least one invalid / edge row
Deliverable	Table with 4+ rows ready to paste into @CsvSource
Reference	exercises/exercise-02-csvsource-table.md

```
$ cat csvsource-table.md
ACTIVE, true, happy path
PROSPECT, false, not yet active
"", false, blank rejected
CLOSED, false, terminal state
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-csvsource-table.md.

ORGANIZING TEST CASES

Well-organized tests are easy to maintain, understand, and scale as your project grows.

KEY PRINCIPLES

- Mirror the main code structure — keep tests under src/test/java following the same package structure as src/main/java.
- A common starting point is one test class per production class — but organize larger suites around behaviors and scenarios when that improves clarity.
- Use descriptive names that express the behavior being tested.
- Keep tests independent — don't depend on execution order.
- Alternatives: nested classes by scenario, separate contract tests, edge-case test classes, or parameterized scenario suites.

NAMING CONVENTIONS

ELEMENT	EXAMPLE
`Test Class`	OrderServiceTest
`Test Method`	should_ReturnOrder_when_ValidId()
`Test Data`	customerWithProspectStatus, duplicateEmail, expiredAccount, maximumAllowedDiscount
`Constants`	MAX_RETRY_COUNT

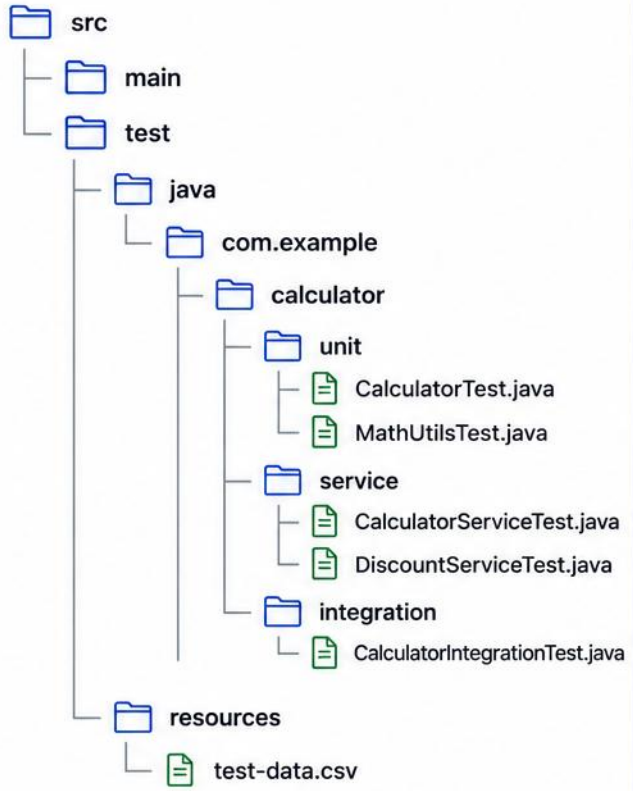
NAMING: ONE CONVENTION AMONG SEVERAL

- should_ReturnOrder_when_ValidId() is one valid convention, not the only correct one.
- Other valid styles: shouldReturnOrderWhenIdIsValid, returnsOrderForValidId, findById_validId_returnsOrder, or @DisplayName("...").
- Teach one team convention — consistency matters more than a single naming formula.

KEY TAKEAWAY A well-structured test suite is easier to maintain, debug, and extend as the codebase grows.

ORGANIZING TEST CASES

PROJECT TEST STRUCTURE



NAMING CONVENTIONS

ELEMENT	CONVENTION	EXAMPLE
Test Class	<ClassUnderTest>Test	CalculatorTest
Test Method	should_<expectedBehavior>_when_<condition>	should_returnSum_when_validInput
Test Data	Descriptive names	validInputs, invalidInputs
Test Packages	By type or feature	unit, integration, service

GROUPING STRATEGIES

BY TEST TYPE

- unit
- integration
- system
- performance

BY FEATURE

Group tests by feature or module being tested.

BY LAYER

Organize tests according to application layers (controller, service, repository, etc.).

BY SCENARIO

Group tests by business scenarios or user workflows.

BEST PRACTICES

- Keep tests independent and isolated.
- One test, one behavior (one assertion focus).
- Use clear, descriptive names.
- Avoid code duplication; use helpers.
- Use test data builders or fixtures.
- Keep tests fast and repeatable.
- Maintain high readability and structure.

A WELL-NAMED, WELL-ORGANIZED TEST CLASS

Naming and structure conventions become obvious once you see them applied in one file.

PRINCIPLE	APPLIED HERE	WHY IT HELPS
1:1 class mapping	OrderServiceTest tests OrderService	Related tests are easy to find
Descriptive names	should_ApplyDiscount_when_OrderExceedsThreshold	Reads like a spec of the behavior
One behavior per test	Each method checks a single outcome	A failure points straight at the cause
Independent setup	@BeforeEach builds a fresh fixture	No test depends on another's leftover state
Mirrors structure	Same package as OrderService, under src/test/java	Easy to navigate as the codebase grows

COMPLETE EXAMPLE (NAMED & ORGANIZED)

```
class OrderServiceTest {

  @Test
  void should_ApplyDiscount_when_OrderExceedsThreshold() {
    OrderService service = new OrderService();
    Order order = new Order(150.00);

    double total = service.priceOf(order);

    assertEquals(135.00, total, 0.001);
  }
}
```

WHY THIS READS WELL

- The class name (OrderServiceTest) makes the class under test obvious at a glance.
- should_<expectedBehavior>_when_<condition> reads as a sentence describing the rule being verified.
- One assertion focus per test means a failing test name alone tells you what broke.

KEY TAKEAWAY A test class that reads like documentation is one your whole team can maintain.

TRY IT YOURSELF — EXERCISE 1: EXPRESSIVE TEST NAMES

Reinforces: naming tests so they read as CRM behavior specs, not method dumps.

STEP	WHAT TO DO
Goal	Write 3 expressive JUnit method names for create/activate CRM cases
File	test-names.md (in examples/module-17-exercises/notes/)
Pattern	method_whenCondition_thenResult — readable in Surefire reports
CRM	Cover CUS-1001 create and CUS-1002 activate scenarios
Deliverable	Three names + one sentence why each beats createCustomerTest
Reference	exercises/exercise-01-test-names.md

```
$ cat test-names.md
createCustomer_whenValid_thenPersistsActive
activateCustomer_whenProspect_thenBecomesActive
createCustomer_whenBlankEmail_thenRejects
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-test-names.md.

DESIGNING EFFECTIVE BUSINESS-LOGIC TESTS

Business logic is the core of your application — cover it with focused scenarios, structured clearly with Arrange-Act-Assert.

Business Rules

- Verify rules, conditions, and calculations.

Boundary Values

- Test min, max, thresholds, and edge cases.

All Scenarios

- Cover positive, negative, and exception cases.

Data Integrity

- Ensure inputs produce valid, consistent outputs.

Regression Safety

- Prevent changes from breaking existing behavior.

MORE SCENARIO TYPES + THE AAA PATTERN

- Also test state transitions (e.g., PROSPECT → ACTIVE) and side-effect verification (e.g., confirm a save or event occurred).
- Use Arrange-Act-Assert when it improves clarity; keep setup, action, and verification easy to distinguish — parameterized and property-based tests may not have visually distinct phases.

FIRST QUALITIES + ONE BEHAVIOR PER TEST

- Fast, Independent, Repeatable, Self-Validating, Timely (write close to the change). One behavior per test — related assertions are fine:

```
assertAll(
    () -> assertEquals(ACTIVE, customer.status()),
    () -> assertNotNull(customer.activatedAt())
);
```

KEY TAKEAWAY Good tests act as living documentation of your business rules — covering scenarios, boundaries, and side effects with clear AAA structure — and give confidence to refactor and extend your code.

TEST COVERAGE CONCEPTS

Test coverage measures how much of your code is exercised by your tests.

COVERAGE TYPE	WHAT IT MEASURES	EXAMPLE
`Statement Coverage`	Percentage of executable statements that are run	Line 1,2,3,4,5 executed
`Branch Coverage`	Percentage of branches (if/else, switch) taken	Both true/false paths executed
`Condition Coverage`	Percentage of boolean conditions evaluated both ways	Each condition true and false
`Path Coverage`	Impractical for complex control flow — use risk-based scenarios, branch/condition coverage instead	Rarely measured directly in practice
`Method / Class Coverage`	Percentage of methods/classes exercised	All methods invoked

NOTE

- Low coverage in critical business logic is a warning sign, but coverage alone does not measure test quality.
- Primary tools: JaCoCo (coverage engine) and IDE coverage (IntelliJ); SonarQube aggregates coverage into broader quality metrics. Cobertura is a legacy/historical tool, not a primary choice today.
- Mutation testing checks whether tests detect deliberately introduced code changes and can reveal weak assertions that line coverage misses.

KEY TAKEAWAY Aim for meaningful tests, not just high numbers — focus on critical business logic, edge cases, and error handling.

TEST COVERAGE

DEFINITION

Test coverage measures the percentage of code executed by tests.

Coverage (%) =

$$\frac{\text{Lines Covered}}{\text{Total Executable Lines}} \times 100$$

COVERAGE LEVELS

0%	0% – No tests executed
1–50%	Low – High risk of defects
51–80%	Medium – Moderate coverage
81–100%	High – Good coverage

COVERAGE TYPES

EXAMPLE

Code

```
if (user.isActive()) {
  if (user.hasPermission()) {
     return "ACCESS";
     } else {
      return "NO_PERMISSION";
  }
} else {
    return "INACTIVE";
}
```

Coverage Report

Line Coverage	:	85%
Branch Coverage	:	75%
Condition Coverage	:	80%
Method Coverage	:	90%

HOW COVERAGE WORKS

Source Code → Run Tests → Coverage Tool (Instrument Code) → Generate Report

BEST PRACTICES

- Aim for meaningful coverage, not 100%
- Focus on critical and business logic
- Track coverage trends over time
- Write tests for happy paths and edge cases
- Use coverage as a quality guideline

COMMON TOOLS

JaCoCo · Cobertura · Istanbul · SonarQube · EclEmma

TRY IT YOURSELF — EXERCISE 4: JACOCO GATE TODOS (STARTER)

Reinforces: coverage as a gate narrative — TODOs, not a finished JaCoCo config.

STEP	WHAT TO DO
Goal	FILL-IN-THE-BLANK starter — narrate a JaCoCo gate for CustomerService
File	jacoco-gate-todos.md (in examples/module-17-exercises/notes/)
Fill in	Target %, packages in scope, what fails the build, AI-review note
CRM	Keep CUS-1001 / CUS-1002 scenarios in the coverage story
Deliverable	Every _____ / TODO filled; honest about what coverage does not prove
Reference	exercises/exercise-04-fill-jacoco-gate-todos.md

```
$ cat jacoco-gate-todos.md
Target: _____ % line coverage on com.northstar.crm.service
Fail build when: _____
Coverage does NOT prove: _____
```

TRY IT NOW Complete Exercise 4 (fill every blank) — see exercises/exercise-04-fill-jacoco-gate-todos.md.

COMMON TESTING MISTAKES

Even experienced developers make testing mistakes — avoiding these pitfalls leads to more reliable tests.

MISTAKE	WHY IT'S A PROBLEM	HOW TO FIX IT
Testing implementation, not behavior	Tests become fragile and break with refactoring	Test the public API and observable behavior — not private implementation details.
Not isolating tests	Tests become slow, flaky, order-dependent	Replace slow/nondeterministic dependencies with the simplest test double (fake, stub, or mock); prefer state-based testing unless interaction verification is part of the behavior.
Poor or missing assertions	Tests may pass even if code is broken	Use meaningful assertions on expected outcomes
Duplicate test code	Harder to maintain, more error-prone	Extract setup that is genuinely shared and clear; keep behavior-specific data inside the test.
Ignoring edge cases	Bugs hide in edge cases (nulls, limits)	Add tests for boundaries and invalid input

KEY TAKEAWAY Good tests are reliable, maintainable, focused on behavior, and provide real confidence.

TESTING BEHAVIOR, NOT IMPLEMENTATION

The most common testing mistake, made concrete: two versions of the same test.

SIGNAL	FRAGILE (AVOID)	ROBUST (PREFER)
What's checked	A private field via reflection	The public method's return value
Coupling	Breaks whenever internals are refactored	Survives internal refactors
Assertion target	Internal counter or list state	Observable result or thrown exception
Isolation	Shares mutable static state across tests	Fresh fixture created in @BeforeEach
Readability	Hard to tell what behavior is verified	Assertion names the expected outcome

BEFORE vs. AFTER

```
// FRAGILE: coupled to a private field
@Test
void activateCustomer_setsFlag() {
    service.activateCustomer("CUS-1001");
    assertEquals(1, service.callCount);
}
// ROBUST: verifies observable behavior
@Test
void should_BecomeActive_when_Prospect() {
    service.activateCustomer("CUS-1001");
    assertEquals(ACTIVE,
        repo.find("CUS-1001").getStatus());
}
```

SPOTTING THE DIFFERENCE

- The fragile test breaks the moment callCount is renamed or removed — even if the behavior is unchanged.
- The robust test asks a business question: is the customer active? It survives any internal refactor.
- Prefer state-based assertions on the public API; reach for interaction verification only when the interaction itself is the behavior.

KEY TAKEAWAY If a refactor with unchanged behavior breaks your tests, the tests were coupled to implementation, not behavior.

TRY IT YOURSELF — EXERCISE 6: LAB 17 PREP CHECKLIST

Reinforces: you are ready for the timed Lab 17 JUnit path.

STEP	WHAT TO DO
Goal	Self-check that pre-lab skills are ready for Lab 17
File	lab17-prep-checklist.md (in examples/module-17-exercises/notes/)
Check	Names, CsvSource, asserts, JaCoCo narrative, AAA plan
Tools	JDK 21 + Maven test goal available
Deliverable	Checklist with Pass/Fail marks — no full lab suite yet
Reference	exercises/exercise-06-lab17-prep-checklist.md

```
$ cat lab17-prep-checklist.md
[ ] Expressive names drafted
[ ] CsvSource table ready
[ ] Meaningful asserts rewritten
[ ] JaCoCo TODOs filled
```

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-lab17-prep-checklist.md.

LAB OVERVIEW – JUNIT TESTING WITH AI ASSISTANCE

Build comprehensive unit tests for a real-world business logic component using JUnit 5 and GitHub Copilot.

WHAT YOU WILL LEARN

- Write effective unit tests using JUnit 5 with assertions, annotations, and test lifecycle.
- Use parameterized tests and measure/improve test coverage.
- Design tests for real business rules, including status transitions and failure paths.

TEST DOUBLES IN THIS LAB (NO MOCKITO YET)

- CustomerService depends on a repository/validator. Do not use Mockito for this lab — mocking frameworks are the focus of Module 18.
- Use an InMemoryCustomerRepository (a simple fake) and deterministic test data instead.
- Collaborator isolation with Mockito is deferred to Module 18: Mockito for Test Isolation.

AI ASSISTANCE (OPTIONAL) — REVIEW CHECKLIST

- Optional: use GitHub Copilot to help brainstorm test ideas or draft stubs. Apply the review practices learned in Modules 10 and 11.
- Before accepting AI-generated tests, check:
- Assertions are meaningful, not just non-null checks.
- Tests fail when production behavior is deliberately broken.
- Edge cases come from requirements, not only AI guesses.
- No implementation-coupled tests (private details, internal fields).
- Generated test names and setup are reviewed for clarity.
- No fabricated APIs or methods that do not exist in the codebase.

KEY TAKEAWAY Achieve meaningful coverage of all documented business rules and critical branches; use 80% line coverage as a diagnostic target, not the sole definition of success.

LAB TASKS AND DELIVERABLES

Build a complete JUnit 5 test suite for a CustomerService, using GitHub Copilot where helpful.

#	TASK	DETAILS	FORMAT
1	Understand the Code	Review the CustomerService class and its business rules.	.java
2	Design Test Cases	Identify scenarios for valid, edge, and invalid inputs.	Notes
3	Write Unit Tests	Implement test cases with proper annotations/assertions.	.java
4	Parameterized Tests	Cover multiple input combinations efficiently.	.java
5	Run & Analyze Coverage	Measure test coverage and identify gaps.	Report
6	AI-Assisted Refactoring	Use GitHub Copilot to review and improve test quality.	Notes

SUCCESS CRITERIA

- Achieve meaningful coverage of all documented business rules and critical branches; use 80% line coverage as a diagnostic target, not the sole definition of success.
- All status transitions are tested; failure paths are tested; assertions are meaningful; no unnecessary tests are written only to increase the percentage.
- Tests are clean, maintainable, and independent of execution order.

KEY TAKEAWAY A complete, well-organized test suite with strong coverage builds lasting confidence in your codebase.

LAB VERIFICATION: BUSINESS RULES & DELIVERABLES

Confirm your CustomerService test suite covers every business rule and can actually catch defects.

SCENARIO	EXPECTED RESULT
Create valid customer	Customer saved
Blank name	Reject
Invalid email	Reject
Duplicate email	Reject
PROSPECT → ACTIVE	Allowed
ACTIVE → PROSPECT	Rejected
Missing customer	Not-found error

PROVE INDEPENDENCE, THEN PROVE DETECTION

- Isolation check: (1) run one test alone, (2) run the full class, (3) reverse/randomize execution order, (4) confirm the results stay the same.
- Red-green check: (1) temporarily break a business rule, (2) run tests and capture the failure, (3) restore the fix, (4) run tests and capture the success.

EXCEPTION ASSERTIONS & DELIVERABLES

- Verify exception type, a stable business code, relevant message/details, and unchanged repository state:

```
CustomerConflictException exception =
    assertThrows(
        CustomerConflictException.class,
        () -> service.activateCustomer("CUS-1001")
    );
assertEquals(
    "INVALID_STATUS_TRANSITION",
    exception.code()
);
```

- Deliverables: test-case matrix, test report, coverage report, a failing-test screenshot, an AI-suggestion note, and a list of uncovered lines with justification.

KEY TAKEAWAY A test suite that is independent, deterministic, and demonstrably catches defects is what makes coverage numbers trustworthy.

JUNIT TEST: DEFINITION OF DONE

Use this checklist to confirm a JUnit test — or a whole suite — is genuinely finished, not just written.

1. Test name describes the behavior.
2. Setup is minimal and readable.
3. Test is independent and deterministic.
4. One behavior is verified.
5. Assertions are meaningful.
6. Happy, boundary, invalid, and exception cases are covered.
7. External dependencies are controlled.
8. Test fails when the behavior is broken.
9. Coverage gaps are understood.
10. No private implementation detail is tested.
11. AI-generated code has been reviewed.
12. Suite runs reliably in IDE and Maven.

KEY TAKEAWAY A test is not done when it compiles and passes once — it is done when it meets this checklist.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of JUnit testing fundamentals.

1. Which annotation marks a method as a test?

A. @TestCase
B. @Test
C. @Check
D. @Run

2. Which assertion verifies that two values are equal?

A. assertEquals(expected, actual)
B. assertTrue(condition)
C. assertSame(expected, actual)
D. assertNotNull(object)

3. What is the purpose of a parameterized test?

A. To run a test only once for performance
B. To run the same test with different input values
C. To test private methods
D. To ignore failing tests

4. Which annotation runs setup code before each test?

A. @BeforeAll
B. @BeforeEach
C. @BeforeTest
D. @Setup

KEY TAKEAWAY JUnit annotations and assertions form the backbone of every well-structured test class.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. Which of these is good practice for a UNIT test (not an integration test)?

A. Test multiple methods in a single test

B. Use real database and external services

C. Use descriptive test names

D. Ignore edge and exception cases

6. A suite has 95% line coverage, but assertions only check that results are non-null. What is the main concern?

A. The suite runs too slowly

B. Weak assertions make high coverage misleading

C. The suite is not using parameterized tests

D. The suite violates the AAA pattern

7. A test passes when run alone but fails after another test modifies shared state. Which principle is violated?

A. Fast

B. Independence / isolation

C. Self-validation

D. Timeliness

8. How can GitHub Copilot help you write better unit tests?

A. By replacing the need for assertions

B. By suggesting test scenarios and explaining failures

C. By skipping edge cases automatically

D. By disabling flaky tests permanently

ANSWER KEY

- **1-B 2-A 3-B 4-B 5-C 6-B 7-B 8-B**

KEY TAKEAWAY Confident, well-tested code starts with clear annotations, meaningful assertions, and disciplined test isolation.

"Well-tested code is easier to change, and easier to trust."

MODULE 17 COMPLETE

JUnit Testing Fundamentals

You now have a strong foundation in JUnit testing — keep practicing to become a confident tester and developer.